String diagrams for Freyd allegories

the graphical calculus of a cartesian bicategory of relations

(Bonchi–Pavlović–Sobociński, *Functorial Semantics for Relational Theories*)

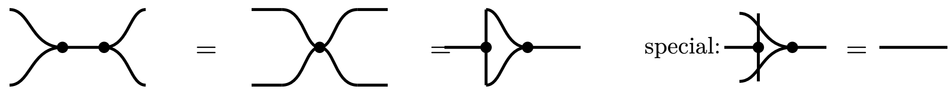
A string diagram is an arrow of a symmetric monoidal category drawn as a circuit: boxes are generators, wires carry objects, **left-to-right** juxtaposition is composition `;` and **vertical** stacking is the monoidal product `⊕`. This is the same reading order as the book's diagram-order composition `x y` = "first `x` then `y`". An allegory's operations — composition, converse `°`, meet `∩`, top `⊤` — are all recovered from **one** extra piece of structure on top of the plain monoidal wires: a **special Frobenius** (co)monoid on every object. Below, black dots are that Frobenius structure; white boxes are relation generators.

1. The generators: the Frobenius structure

Four nodes, all drawn with the same black dot because they form **one** special Frobenius algebra (the "spider"): any connected diagram of dots collapses to a single node fixed only by its input/output count.

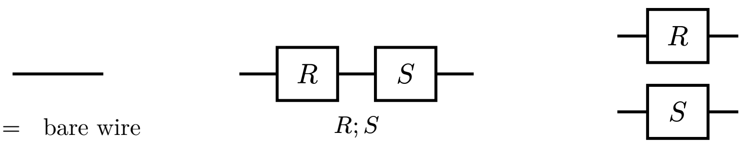
copy $\Delta : 1 \to 2$ discard $! : 1 \to 0$ merge $\nabla : 2 \to 1$ unit $? : 0 \to 1$

In **Rel** these are the **diagonal** relations: `copy` is $x \mapsto (x, x)$, `discard` is $x \mapsto \checkmark$ (the map to a point), `merge = copy°`, `unit = discard°`. `(Δ, !)` is a cocommutative comonoid, `(∇, ?)` a commutative monoid, and together they satisfy the Frobenius law and the **special** law `Δ;∇ = id`:



2. Identity, composition, monoidal product

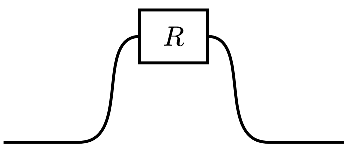
A relation `R : m → n` is a labelled box. The identity is a bare wire; `R ; S` (first `R` then `S`) is horizontal juxtaposition; `R ⊕ S` is vertical stacking.



id = bare wire $R; S$ $R \oplus S$

3. Converse `R°` — bending the wire

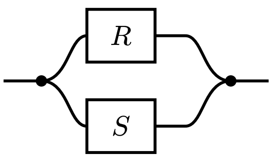
The Frobenius structure makes the category **compact closed**: every wire can be bent back with a `cup` (`? ; Δ`, an "unfork") and a `cap` (`∇ ; !`). Converse is then **free** — you just bend both of `R`'s wires around, so what was an input is read as an output:

$R^\circ :=$ 

in **Rel**: $x \, R^\circ \, y \Longleftrightarrow y \, R \, x$

4. Meet `R ∩ S` — the convolution (copy · pair · merge)

Intersection is **not** a new generator: it is `Δ ; (R ⊕ S) ; ∇`. Copy the input, run `R` and `S` on the two copies, then merge — the merge forces the two outputs to **coincide**, i.e. demands both `x R y` **and** `x S y`. This is why "everything in parallel is conjunction": a signal flows through **all** vertical branches at once (a **wave**).

$R \cap S :=$ 

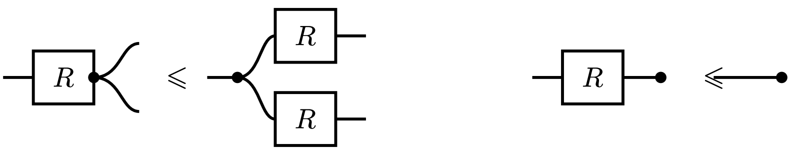
The unit of `∩` is the **top** relation `⊤ = ! ; ?` (discard everything, then create everything — the all-pairs relation):

$\top :=$

With `;`, `°`, `∩`, `⊤` in hand, the **modular law** `R S ∩ T ≤ R(S ∩ R°T)` is a **derivable** theorem of this calculus — so a plain string diagram over these generators is exactly a term of a (unitary, tabular) **allegory**.

5. When is a relation a map? — the defining inequations

Every relation is only a **lax** comonoid homomorphism: copying then applying is `≥` applying then copying. Equality on the two laws below says precisely **single-valued** and **total** — i.e. `R` is the graph of a function (a `map`, the arrows the book calls covers/maps).



single-valued (= at equality) total (= at equality)

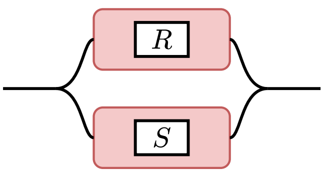
6. What cannot be drawn: union `∪` and empty `⊥`

Tarski's calculus of relations has **two** families of operations:

conjunctive (drawable):	$R; S$ R° $R \cap S$ $\top$ id
disjunctive (**not** drawable):	$R \cup S$ $\bot$

Intersection sneaks in through the Frobenius structure because it reuses the **same** monoidal product `⊕` (copy on one product, merge on the same one). Union is a **genuinely second** monoidal product: in **Rel** it is the **biproduct** `⊕` (disjoint union of objects, block-diagonal on arrows), over which `;` distributes — making **Rel** a **rig category** with two tensors. A string diagram is the language of **one** monoidal category, so it has no room to express the second product. Formally, each hom of a cartesian bicategory of relations is only a **meet-semilattice** — there is provably no `∪` and no `⊥`.

The intuition: inside a circuit a signal is a **wave**, passing through **all** vertical branches simultaneously → conjunction. Union needs a signal that is a **particle**, taking **one** branch → choice. You cannot have both on one sheet. The fix (Bonchi–Di Giorgio–Santamaria, **tape diagrams**) is a second layer — "string diagrams of string diagrams": whole circuits `R`, `S` are wrapped in a **tape** that forks, and a particle flows through exactly one:

$R \cup S :=$ 

a **tape**: the particle takes the R branch <u>or</u> the S branch

Everything above the pink tapes is an ordinary allegory (an `∩`-calculus, drawable). Adding the tapes gives a **distributive** allegory (`∪`, `⊥`). Higher still — **division** allegories (residuals `R\\S`, right adjoints to `;`) and **power** allegories (the `∧` power-object transpose) — are not finite string-diagram terms at all: they are adjoints / closed structure, not composites of the generators.